

HANDS-ON ACTIVITY FOCUS GUIDE

Map and Verify the Motors

From Java names to real FTC hardware

25 MIN ACTIVITY	8 STUDENT ROLES	LIVE MOTOR TEST FORMAT	1 SAFETY LEAD
---------------------------	---------------------------	----------------------------------	-------------------------

ACTIVITY OUTCOME Students distinguish declaring a motor from mapping it, verify exact configuration names, and perform a controlled direction test without approaching an enabled drivetrain.

Materials

- Robot secured on stable blocks with both drive wheels fully clear.
- Robot Controller configuration and BasicTwoMotorTeleOp.java displayed side by side.
- Printed mapping and direction-observation sheet from this guide.
- Gamepad and Driver Station; POWERING ON safety poster visible at the test boundary.
- Optional printed mismatch cards: left_motor, leftMotor, Left_Motor, and left motor.

Before students enter

- Confirm the battery, wiring, wheel retention, and robot stand are secure.
- Verify the OpMode can initialize with the intended configuration before class.
- Choose the team's physical forward direction and mark it on the robot with tape.
- Set a boundary that nobody crosses while the OpMode is enabled.

SAFETY BOUNDARY Students never cross into the robot test area while the OpMode is enabled. STOP and visually confirm DISABLED before anyone approaches.

Student setup and roles

Assign one primary role to each student. Rotate roles on a second trial if time permits. The safety lead may stop the activity at any time.

STUDENT	ROLE	RESPONSIBILITY
1	Safety lead	Reads each POWERING ON step and watches the boundary.
2	Driver Station	Selects INIT, PLAY, and STOP only when directed.
3	Code reader	Finds declarations and hardwareMap.get() lines.
4	Config navigator	Reads the configured device names exactly.
5	Left observer	Records left-wheel direction from outside the boundary.
6	Right observer	Records right-wheel direction from outside the boundary.
7	Recorder	Completes the mapping and direction tables.
8	Debugger	Proposes one evidence-based correction after STOP.

Ground rules

- Predict before testing; record evidence before proposing a correction.
- Only the Driver Station operator touches INIT, PLAY, or STOP.
- Any student may call STOP for unexpected motion, instability, heat, noise, or an unclear boundary.
- Change only one variable between tests so the evidence remains interpretable.

POWERING ON gate

SAY AND DO ALL SIX STEPS 1. Secure the robot. 2. Confirm DISABLED. 3. Look around. 4. Make sure everyone is clear. 5. Call POWERING ON! and wait. 6. Visually confirm clear, then enable/PLAY.

After every test: press STOP, visually confirm DISABLED, and only then permit anyone to approach the robot.

25-minute teacher walkthrough

1. Separate declaration from connection | 4 minutes

TEACHER	<i>"Point to private DcMotor leftMotor;. Say: This creates a typed Java name. Has Java connected to the real motor yet?"</i>
---------	--

STUDENT	<i>"The code reader identifies DcMotor as the type and leftMotor as the variable. The class answers no connection exists yet."</i>
---------	--

ASK	<i>"What new job does hardwareMap.get() perform?"</i>
-----	---

LISTEN FOR It looks up a configured device and assigns that hardware object to the Java variable.

TEACHING NOTE Use a blank name tag as the analogy: the label exists before it is attached to a person or object.

2. Run an exact-name audit | 5 minutes

TEACHER	<i>"Read each hardwareMap.get() call from the inside out: device type, quoted configuration name, returned motor, assigned variable."</i>
---------	---

STUDENT	<i>"The config navigator reads the Robot Controller name aloud while the recorder compares every character, underscore, and capital."</i>
---------	---

ASK	<i>"Would leftMotor match left_motor?"</i>
-----	--

LISTEN FOR No. The Java variable may be named leftMotor, but the quoted configuration string must exactly match left_motor.

TEACHING NOTE Keep the Java variable name and configuration string in separate columns so students do not blend them together.

3. Predict a mismatch | 4 minutes

TEACHER	<i>"Show one incorrect name card. Ask students to predict when the failure occurs and whether PLAY should be pressed."</i>
---------	--

STUDENT	<i>"The debugger marks the mismatch, predicts an initialization error, and keeps the test at DISABLED."</i>
---------	---

ASK	<i>"Why is this not a motor-direction problem?"</i>
-----	---

LISTEN FOR	The program cannot find the device, so mapping fails before the driving loop can command it.
------------	--

TEACHING NOTE	A printed error challenge is enough. Do not intentionally alter a working robot configuration unless the teacher wants that extension.
---------------	--

4. Perform the first direction test | 8 minutes

TEACHER	<i>"Robot secured. Confirm DISABLED. Look around. Make sure everyone is clear. Call POWERING ON! Wait. Visually confirm clear. Then direct the operator to INIT and PLAY."</i>
---------	--

STUDENT	<i>"The driver gently moves only the left stick, centers it, then only the right stick. Observers record which way the top of each wheel moves relative to the forward arrow."</i>
---------	--

ASK	<i>"What happens immediately if motion differs from the prediction?"</i>
-----	--

LISTEN FOR	Press STOP, confirm DISABLED, and keep hands away until the disabled state is verified.
-------------------	---

TEACHING NOTE	Test one side at a time first. This isolates mapping and direction evidence.
----------------------	--

5. Diagnose one variable at a time | 4 minutes

TEACHER	<i>"After STOP and DISABLED, compare commanded side, observed side, configuration name, and direction setting. Change only one cause before retesting."</i>
---------	---

STUDENT	<i>"The debugger states the evidence and proposes either a name correction or a direction-setting correction. The team repeats the complete safety sequence before another PLAY."</i>
---------	---

ASK	<i>"Is the right motor always the side that must be reversed?"</i>
-----	--

LISTEN FOR	No. The required side depends on mounting, gearing, wiring, and the team's forward convention.
-------------------	--

TEACHING NOTE	Do not let students swap wires, touch wheels, or edit a live system while enabled.
----------------------	--

Motor mapping and direction record

Student/team: _____ Date: _____ Robot: _____

CHECK	LEFT SIDE	RIGHT SIDE	EVIDENCE / NOTES
Java variable	leftMotor	rightMotor	
Device type	DcMotor.class	DcMotor.class	
Quoted code name			Copy exactly
Robot Configuration name			Copy exactly
Exact match?	Yes / No	Yes / No	Circle differences
Gentle forward-stick test	Wheel: ____	Wheel: ____	Expected: ____
Direction correction needed?	Yes / No	Yes / No	Only after DISABLED
STUDENT EVIDENCE RULE Write what was commanded and what was observed. Do not write a correction until STOP has been pressed and DISABLED has been confirmed.			

Safety sign-off

Before live testing, the safety lead confirms: robot secure ☐ DISABLED ☐ boundary clear ☐ POWERING ON called and wait completed ☐

After testing: STOP pressed ☐ DISABLED visually confirmed ☐ Safety lead initials: _____

COMPLETION CRITERIA The student uses lesson vocabulary, predicts before testing, records evidence, explains one result from the code or math, and follows STOP/DISABLED rules before approaching the robot.

Debrief

ASK	LISTEN FOR
What is created by <code>private DcMotor leftMotor;?</code>	A field/reference that can hold a DcMotor object; it is not yet mapped.
What are the two key <code>hardwareMap.get()</code> arguments?	The device type and the exact Robot Configuration name string.
Why do we test one side at a time?	It isolates which command is connected to which wheel and makes direction evidence easier to interpret.
What must happen before anyone approaches after a test?	STOP must be pressed and DISABLED must be visually confirmed.

Troubleshooting without guessing

OBSERVATION	NEXT CHECK AFTER STOP/DISABLED
INIT reports device not found	Compare quoted names character by character with Robot Configuration.
Left stick moves right wheel	Check which hub port and configured name represent each physical motor.
Both respond but one wheel is physically backward	After STOP/DISABLED, review that motor's Direction setting.
A wheel does not move	STOP; check telemetry command, configuration, wiring, battery, and mechanical binding one at a time.

Technical notes for the teacher

- A declaration creates a typed reference; `hardwareMap.get()` retrieves the configured hardware object.
- Configuration strings are exact identifiers. Case, spaces, and underscores matter.
- Commented direction commands do nothing until the leading `//` is removed.
- Direction correction is installation-specific; do not teach that one universal side must be reversed.
- The first test should use gentle input with the wheels clear of the table.

EXIT TICKET Explain the difference between `leftMotor` and `"left_motor"` in one or two sentences.

HANDS-ON ACTIVITY FOCUS GUIDE

Observe INIT, PLAY, and STOP

Follow execution through the Driver Station states

25 MIN ACTIVITY	8 STUDENT ROLES	STATE OBSERVATION FORMAT	1 SAFETY LEAD
---------------------------	---------------------------	------------------------------------	-------------------------

ACTIVITY OUTCOME Students identify where execution is located during INIT, explain why the robot does not drive before PLAY, and show how STOP makes the active condition false.

Materials

- Robot secured on blocks; gamepad and Driver Station connected.
- BasicTwoMotorTeleOp.java displayed with telemetry and waitForStart() visible.
- Printed state-observation sheet from this guide.
- Cards labeled BEFORE INIT, INITIALIZED, WAITING, ACTIVE, and STOPPED.
- POWERING ON safety poster displayed beside the robot area.

Before students enter

- Confirm Status: Initialized is present before waitForStart() in the source code.
- Verify both wheels are clear and define the no-entry boundary.
- Practice locating the Driver Station indicators for INIT, PLAY, STOP, and disabled state.
- Keep joystick input centered before beginning.

SAFETY BOUNDARY Students never cross into the robot test area while the OpMode is enabled. STOP and visually confirm DISABLED before anyone approaches.

Student setup and roles

Assign one primary role to each student. Rotate roles on a second trial if time permits. The safety lead may stop the activity at any time.

STUDENT	ROLE	RESPONSIBILITY
1	Safety lead	Controls the call-and-clear sequence.
2	Driver Station	Presses INIT, PLAY, and STOP when authorized.
3	Code tracker	Points to the instruction where execution is located.
4	Telemetry reader	Reads each displayed telemetry message aloud.
5	Wait gate	Blocks progress until PLAY is pressed.
6	Joystick tester	Moves one stick only when the teacher authorizes it.
7	Recorder	Completes the state-observation table.
8	Stop verifier	Confirms the OpMode ended and the robot is DISABLED.

Ground rules

- Predict before testing; record evidence before proposing a correction.
- Only the Driver Station operator touches INIT, PLAY, or STOP.
- Any student may call STOP for unexpected motion, instability, heat, noise, or an unclear boundary.
- Change only one variable between tests so the evidence remains interpretable.

POWERING ON gate

SAY AND DO ALL SIX STEPS 1. Secure the robot. 2. Confirm DISABLED. 3. Look around. 4. Make sure everyone is clear. 5. Call POWERING ON! and wait. 6. Visually confirm clear, then enable/PLAY.

After every test: press STOP, visually confirm DISABLED, and only then permit anyone to approach the robot.

25-minute teacher walkthrough

1. Predict four states | 4 minutes

TEACHER	<i>"Place the four state cards in order. Ask students to predict whether motor commands can repeat in each state."</i>
---------	--

STUDENT	<i>"The recorder writes predictions for before INIT, after INIT, after PLAY, and after STOP."</i>
---------	---

ASK	<i>"Which event releases waitForStart()?"</i>
-----	---

LISTEN FOR PLAY releases the wait; STOP can also cause the wait to return without making the OpMode active.

TEACHING NOTE Make students commit to predictions before touching the Driver Station.

2. Press INIT and locate execution | 6 minutes

TEACHER	<i>"With the robot disabled and secured, select the OpMode and press INIT. Ask the code tracker to follow runOpMode() until execution pauses."</i>
---------	--

STUDENT	<i>"The code tracker points through hardware mapping and telemetry, then stops at waitForStart(). The telemetry reader reads Status: Initialized."</i>
---------	--

ASK	<i>"Does Initialized mean the control loop is running?"</i>
-----	---

LISTEN FOR No. Setup reached the waiting point, but PLAY has not released execution into active control.

TEACHING NOTE If initialization fails, stay disabled and return to Lesson 2's mapping checks.

3. Test the waiting gate | 4 minutes

TEACHER	<i>"Ask the joystick tester to move a stick before PLAY while everyone watches from outside the boundary."</i>
---------	--

STUDENT	<i>"Students confirm that no active-loop motor response occurs. The wait-gate student continues to block the code path."</i>
---------	--

ASK	<i>"Why can the gamepad move without moving the wheel?"</i>
-----	---

LISTEN FOR The statements that read the stick and call `setPower()` are after `waitForStart()` and are not executing yet.

TEACHING NOTE Do not confuse no motion with proof of complete safety; the robot remains treated as energized equipment.

4. Release the gate with PLAY | 7 minutes

TEACHER	<i>"Run the complete POWERING ON sequence. After the pause and visual clear check, authorize PLAY and one gentle stick movement."</i>
---------	---

STUDENT	<i>"The wait gate opens. The code tracker moves to opModelsActive() and the loop. The joystick tester makes one gentle command and returns to center."</i>
---------	--

ASK	<i>"What changed in the program at PLAY?"</i>
-----	---

LISTEN FOR	waitForStart() returned, the active condition became true, and loop instructions could execute.
------------	---

TEACHING NOTE	Use only enough input to make the state change visible.
---------------	---

5. Press STOP and prove the exit | 4 minutes

TEACHER	<i>"Direct the Driver Station operator to press STOP. Ask the stop verifier for two pieces of evidence."</i>
---------	--

STUDENT	<i>"The verifier confirms the wheel stops and the Driver Station shows the OpMode is no longer active/robot is disabled. The code tracker leaves the loop."</i>
---------	---

ASK	<i>"What value does opModelsActive() have now?"</i>
-----	---

LISTEN FOR	False, so another loop iteration does not begin.
------------	--

TEACHING NOTE	Confirm DISABLED before anyone crosses the boundary.
---------------	--

Program-state observation record

Student/team: _____ Date: _____ Robot: _____

STATE	CODE LOCATION	MOTOR LOOP?	OBSERVED EVIDENCE
Before INIT	Not running	No	
After INIT	Setup -> waitForStart()	No	
Stick moved before PLAY	Still waiting	No	
After PLAY	Active check / while loop	Yes	
Stick returned to center	Loop continues	Yes; power may be 0	
After STOP	Loop exited	No	
DISABLED confirmed	OpMode ended	No	Initials: ____
STUDENT EVIDENCE RULE Write what was commanded and what was observed. Do not write a correction until STOP has been pressed and DISABLED has been confirmed.			

Safety sign-off

Before live testing, the safety lead confirms: robot secure ☐ DISABLED ☐ boundary clear ☐ POWERING ON called and wait completed ☐

After testing: STOP pressed ☐ DISABLED visually confirmed ☐ Safety lead initials: _____

COMPLETION CRITERIA The student uses lesson vocabulary, predicts before testing, records evidence, explains one result from the code or math, and follows STOP/DISABLED rules before approaching the robot.

Debrief

ASK	LISTEN FOR
Why can Initialized be displayed before the robot can drive?	The telemetry was sent before waitForStart(); execution is paused until PLAY.
What is the difference between centered sticks and STOP?	Centered sticks request zero power while the loop still runs; STOP ends the OpMode.
What does telemetry.update() do?	It transmits the current telemetry packet to the Driver Station.
What proves it is safe to approach?	The team presses STOP and visually confirms DISABLED; stopped wheels alone are not enough.

Troubleshooting without guessing

OBSERVATION	NEXT CHECK AFTER STOP/DISABLED
No Initialized message	Check whether mapping failed and whether telemetry.update() was reached.
Wheel moves before PLAY	Press STOP immediately; verify the selected code and investigate unexpected hardware behavior.
PLAY pressed but no response	Check active state, gamepad connection, telemetry, mapping, and commands in that order.
Wheel stops but OpMode remains active	Recognize a centered/zero command; use STOP to end the OpMode.

Technical notes for the teacher

- The FTC runtime invokes runOpMode() during initialization for a LinearOpMode.
- waitForStart() blocks until START or until a stop request causes it to return.
- telemetry.addData() prepares an item; telemetry.update() sends the current packet.
- The outer active check prevents loop entry if STOP was requested before PLAY.
- After STOP, opModelsActive() is false and the runtime disables hardware outputs.

EXIT TICKET Why can Status: Initialized be true while joystick movement still produces no wheel response?

HANDS-ON ACTIVITY FOCUS GUIDE

Be the Control Loop

Check, read, command, report, and repeat

22 MIN ACTIVITY	8 STUDENT ROLES	UNPLUGGED FIRST FORMAT	1 SAFETY LEAD
---------------------------	---------------------------	----------------------------------	-------------------------

ACTIVITY OUTCOME Students trace complete while-loop iterations, use fresh inputs on each cycle, and exit immediately when the active condition becomes false.

Materials

- Printed BasicTwoMotorTeleOp.java with space to mark braces.
- Role cards: CHECK, READ, COMMAND, REPORT, DRIVER, CODE TRACER, and STOP.
- Program token; TRUE/FALSE cards; three left/right joystick-value cards.
- Optional secured robot and telemetry display for the final extension.
- POWERING ON safety poster if the live extension will be used.

Before students enter

- Mark the while-loop opening and closing braces on your teacher copy.
- Prepare input pairs such as +0.4/+0.4, +0.6/+0.2, and 0.0/0.0.
- Arrange CHECK -> READ -> COMMAND -> REPORT in a loop with STOP outside it.
- Keep the live robot disabled; the core activity requires no motor power.

SAFETY BOUNDARY Students never cross into the robot test area while the OpMode is enabled. STOP and visually confirm DISABLED before anyone approaches.

Student setup and roles

Assign one primary role to each student. Rotate roles on a second trial if time permits. The safety lead may stop the activity at any time.

STUDENT	ROLE	RESPONSIBILITY
1	CHECK	Reads opModelsActive() and chooses repeat or exit.
2	READ	Accepts the newest joystick-value card.
3	COMMAND	Sends the current values to the motors.
4	REPORT	Announces the telemetry values.
5	Driver	Changes the input card between cycles.
6	Code tracer	Points to each represented code line.
7	STOP	Receives the token when the condition is false.
8	Recorder	Documents each completed iteration.

Ground rules

- Predict before testing; record evidence before proposing a correction.
- Only the Driver Station operator touches INIT, PLAY, or STOP.
- Any student may call STOP for unexpected motion, instability, heat, noise, or an unclear boundary.
- Change only one variable between tests so the evidence remains interpretable.

POWERING ON gate

SAY AND DO ALL SIX STEPS 1. Secure the robot. 2. Confirm DISABLED. 3. Look around. 4. Make sure everyone is clear. 5. Call POWERING ON! and wait. 6. Visually confirm clear, then enable/PLAY.

After every test: press STOP, visually confirm DISABLED, and only then permit anyone to approach the robot.

22-minute teacher walkthrough

1. Find the loop boundary | 4 minutes

TEACHER	"Ask students to box the condition and connect the opening and closing braces of the while statement."
---------	--

STUDENT	"The code tracer outlines exactly which statements belong to the loop body."
---------	--

ASK	"Do the braces repeat?"
-----	-------------------------

LISTEN FOR The statements inside the braces repeat; the condition is checked before each attempted iteration.

TEACHING NOTE Students often include the closing code or miss telemetry. Make them trace brace to matching brace.

2. Compare if and while | 4 minutes

TEACHER	"Point to the outer if and inner while. Ask how many times each condition is evaluated when execution reaches it."
---------	--

STUDENT	"CHECK demonstrates: if chooses once; while checks again before every cycle."
---------	---

ASK	"Why is the outer if redundant here?"
-----	---------------------------------------

LISTEN FOR The while checks the same active condition before entering and before every repetition.

TEACHING NOTE Keep the outer if in this source walkthrough; refactoring can come after students understand the behavior.

3. Trace three fresh cycles | 8 minutes

TEACHER	"Give the driver a different input pair before each cycle. Pass the token CHECK -> READ -> COMMAND -> REPORT -> CHECK."
---------	---

STUDENT	"Students announce condition, input, motor command, and telemetry. The recorder completes one row per cycle."
---------	---

ASK	"Why must READ happen again?"
-----	-------------------------------

LISTEN FOR	The driver's hands may have moved; responsive TeleOp uses the newest gamepad values.
-------------------	--

TEACHING NOTE	Do not let the token return to IDENTIFY, CONNECT, or WAIT. Those startup actions do not repeat.
----------------------	---

4. Exit on FALSE | 3 minutes

TEACHER	<i>"Replace TRUE with FALSE when the token returns to CHECK. Ask what happens before another READ."</i>
---------	---

STUDENT	<i>"CHECK passes the token directly to STOP. READ must not accept another input card."</i>
---------	--

ASK	<i>"Does the loop finish one more cycle after FALSE?"</i>
-----	---

LISTEN FOR	No. The condition is checked first, so the body does not execute again.
-------------------	---

TEACHING NOTE	The immediate stillness after STOP makes the exit condition concrete.
----------------------	---

5. Optional telemetry transfer | 3 minutes

TEACHER	<i>"If using the robot, secure it and run the complete POWERING ON sequence before PLAY. Keep sticks centered and observe telemetry changing with a gentle input."</i>
---------	--

STUDENT	<i>"Students name the human role represented by each changing value, then the operator presses STOP and confirms DISABLED."</i>
---------	---

ASK	<i>"Which role produces the displayed values?"</i>
-----	--

LISTEN FOR	REPORT/telemetry displays the values computed and commanded during that cycle.
-------------------	--

TEACHING NOTE	Skip live power if time, staffing, or robot readiness is not adequate.
----------------------	--

Control-loop cycle record

Student/team: _____ Date: _____ Robot: _____

CYCLE	ACTIVE?	NEW INPUT L / R	COMMAND / REPORT / NEXT
1	TRUE	+0.4 / +0.4	Motors: ____ Telemetry: ____ Next: ____
2	TRUE	+0.6 / +0.2	Motors: ____ Telemetry: ____ Next: ____
3	TRUE	0.0 / 0.0	Motors: ____ Telemetry: ____ Next: ____
4	FALSE	New card withheld	Does loop body run? ____ Next: ____
Student-created	____	____ / ____	Prediction: _____
STUDENT EVIDENCE RULE Write what was commanded and what was observed. Do not write a correction until STOP has been pressed and DISABLED has been confirmed.			

Safety sign-off

Before live testing, the safety lead confirms: robot secure [] DISABLED [] boundary clear [] POWERING ON called and wait completed []

After testing: STOP pressed [] DISABLED visually confirmed [] Safety lead initials: _____

COMPLETION CRITERIA The student uses lesson vocabulary, predicts before testing, records evidence, explains one result from the code or math, and follows STOP/DISABLED rules before approaching the robot.

Debrief

ASK	LISTEN FOR
Which jobs repeat?	CHECK, READ, COMMAND, and REPORT repeat while the condition remains true.
Why is an active check placed before the body?	It prevents another body execution after STOP or another stop request.
What would happen if inputs were read only once?	The robot would keep using stale values and feel unresponsive to later stick movement.
Does motor power 0.0 end the loop?	No. It requests stopped motors while the OpMode can remain active.

Troubleshooting without guessing

OBSERVATION	NEXT CHECK AFTER STOP/DISABLED
Token returns to startup	Redirect REPORT to CHECK, then TRUE sends it to READ.
Students skip telemetry	Require REPORT before the token can return to CHECK.
FALSE still performs READ	Place FALSE physically between CHECK and STOP; body actions are bypassed.
Live values appear unchanged	Confirm the gamepad is connected and that telemetry.update() occurs inside the loop.

Technical notes for the teacher

- `opModelsActive()` becomes true after START and remains true until a stop is requested.
- A while condition is evaluated before each attempted iteration.
- The supplied outer if is redundant because the while checks the same condition.
- The loop has no explicit delay and runs as quickly as its operations and runtime permit.
- Fresh gamepad reads on every iteration are what make driver control responsive.

EXIT TICKET Describe one full loop cycle and explain exactly what prevents the next cycle after STOP.

HANDS-ON ACTIVITY FOCUS GUIDE

The Human Minus Sign

Turn raw joystick Y values into motor-power requests

20 MIN ACTIVITY	8 STUDENT ROLES	MATH + MOTION FORMAT	1 SAFETY LEAD
---------------------------	---------------------------	--------------------------------	-------------------------

ACTIVITY OUTCOME Students read assignment statements from right to left, negate FTC Y-axis values, and connect the stored result to a predicted tank-drive command.

Materials

- Role cards: RAW VALUE, MINUS SIGN, POWER VARIABLE, and MOTION PREDICTOR.
- Number cards: -1.0, -0.7, -0.2, 0.0, +0.35, +0.8, and +1.0.
- Printed joystick-conversion sheet from this guide.
- BasicTwoMotorTeleOp.java displayed at the leftPower/rightPower assignments.
- Optional secured robot with power telemetry visible.

Before students enter

- State the lesson convention: positive stored power is intended as robot-forward after direction configuration.
- Keep raw gamepad values separate from stored power values by using different colored cards.
- Prepare both single-value and left/right tank-drive examples.
- If using the robot, secure it and keep it disabled until the final extension.

SAFETY BOUNDARY Students never cross into the robot test area while the OpMode is enabled. STOP and visually confirm DISABLED before anyone approaches.

Student setup and roles

Assign one primary role to each student. Rotate roles on a second trial if time permits. The safety lead may stop the activity at any time.

STUDENT	ROLE	RESPONSIBILITY
1	Left raw value	Displays gamepad1.left_stick_y.
2	Right raw value	Displays gamepad1.right_stick_y.
3	Left minus sign	Changes the sign of the left raw value.
4	Right minus sign	Changes the sign of the right raw value.
5	Left power	Holds the result stored in leftPower.
6	Right power	Holds the result stored in rightPower.
7	Motion predictor	Predicts forward, backward, stop, curve, or spin.
8	Recorder	Writes raw values, stored values, and predictions.

Ground rules

- Predict before testing; record evidence before proposing a correction.
- Only the Driver Station operator touches INIT, PLAY, or STOP.
- Any student may call STOP for unexpected motion, instability, heat, noise, or an unclear boundary.
- Change only one variable between tests so the evidence remains interpretable.

POWERING ON gate

SAY AND DO ALL SIX STEPS 1. Secure the robot. 2. Confirm DISABLED. 3. Look around. 4. Make sure everyone is clear. 5. Call POWERING ON! and wait. 6. Visually confirm clear, then enable/PLAY.

After every test: press STOP, visually confirm DISABLED, and only then permit anyone to approach the robot.

20-minute teacher walkthrough

1. Read assignment right to left | 4 minutes

TEACHER	<i>"Display double leftPower = -gamepad1.left_stick_y;. Start at the right side and narrate the order of work."</i>
---------	---

STUDENT	<i>"Students identify the raw input, apply unary minus, and place the result into leftPower. They identify double as a decimal-number type."</i>
---------	--

ASK	<i>"Which side is evaluated first?"</i>
-----	---

LISTEN FOR The expression on the right; its result is assigned to the variable on the left.

TEACHING NOTE The equals sign means assignment here, not a question asking whether two things are equal.

2. Become the sign changer | 5 minutes

TEACHER	<i>"Hand RAW VALUE -0.7 to a student. Ask MINUS SIGN to transform it without changing its magnitude."</i>
---------	---

STUDENT	<i>"RAW VALUE shows -0.7; MINUS SIGN announces negative of negative 0.7; POWER VARIABLE displays +0.7."</i>
---------	---

ASK	<i>"What happens to 0.0?"</i>
-----	-------------------------------

LISTEN FOR Its negation is still 0.0; zero has no forward/backward direction.

TEACHING NOTE Say negate or change the sign, not make it negative. A positive raw value becomes negative.

3. Process a tank-drive pair | 6 minutes

TEACHER	<i>"Give the left and right raw students different cards. Both paths operate at the same time conceptually, but calculate each path clearly."</i>
STUDENT	<i>"Each minus-sign student transforms a raw value; power students display results; the motion predictor compares left and right."</i>
ASK	<i>"If stored powers are +0.7 and +0.3, what is predicted?"</i>
LISTEN FOR Both sides request forward, but the left side is stronger, so the robot should curve toward the slower right side.	
TEACHING NOTE Physical direction still depends on correct motor mapping and direction configuration.	

4. Separate zero power from STOP | 2 minutes

TEACHER	<i>"Use raw 0.0 on both sticks. Ask whether the OpMode ends."</i>
---------	---

STUDENT	<i>"Both power students show 0.0. The predictor says motors are commanded to stop, but the loop remains active."</i>
---------	--

ASK	<i>"What action actually ends TeleOp?"</i>
-----	--

LISTEN FOR The driver presses STOP, making the active condition false.

5. Optional telemetry check | 3 minutes

TEACHER	<i>"Secure the robot and complete the POWERING ON sequence. Move one stick gently and compare raw direction, stored sign, and telemetry."</i>
---------	---

STUDENT	<i>"Students state the expected sign before reading the telemetry, then STOP and confirm DISABLED."</i>
---------	---

ASK	<i>"Why may the released stick show a tiny nonzero value?"</i>
-----	--

LISTEN FOR Real joysticks can drift near center; a later program can apply a deadband.

TEACHING NOTE Do not introduce deadband code until students can trace the original expression.

Joystick sign-conversion practice

Student/team: _____ Date: _____ Robot: _____

RAW Y VALUE	APPLY -Y	STORED POWER	PREDICTED REQUEST
-1.0	$-(-1.0)$		
-0.7	$-(-0.7)$		
0.0	$-(0.0)$		
+0.35	$-(+0.35)$		
+1.0	$-(+1.0)$		
Student value: ____			
STUDENT EVIDENCE RULE Write what was commanded and what was observed. Do not write a correction until STOP has been pressed and DISABLED has been confirmed.			

Safety sign-off

Before live testing, the safety lead confirms: robot secure [] DISABLED [] boundary clear [] POWERING ON called and wait completed []

After testing: STOP pressed [] DISABLED visually confirmed [] Safety lead initials: _____

COMPLETION CRITERIA The student uses lesson vocabulary, predicts before testing, records evidence, explains one result from the code or math, and follows STOP/DISABLED rules before approaching the robot.

Debrief

ASK	LISTEN FOR
Why is Y negated?	FTC gamepad Y is normally negative when pushed forward; negation creates a convenient positive-forward command.
What does double communicate?	The variable stores a floating-point/decimal numeric value.
Are leftPower and rightPower permanent fields here?	No. They are local variables created during each loop iteration.
Does +0.5 telemetry guarantee physical forward motion?	No. It is the command; physical direction also depends on configuration and mounting.

Troubleshooting without guessing

OBSERVATION	NEXT CHECK AFTER STOP/DISABLED
Student says minus always makes negative	Use +0.4 and -0.4 side by side; unary minus reverses either sign.
Student reads assignment left to right	Cover the left side, calculate the right expression, then reveal the destination.
Predicted curve is reversed	Identify which side is slower; a differential drive curves toward the slower side.
Telemetry is slightly nonzero at center	Recognize gamepad drift; note deadband as a later improvement.

Technical notes for the teacher

- FTC joystick axes typically report floating-point values from -1.0 to +1.0.
- Pushing a Y axis forward normally produces a negative raw value.
- Unary minus is applied before the result is assigned to the power variable.
- The power variables in this program are recreated on every loop iteration.
- Negating input does not replace correct motor direction configuration.

EXIT TICKET If right_stick_y is +0.35, what is stored in rightPower, and what motion does that side request under the lesson convention?

HANDS-ON ACTIVITY FOCUS GUIDE

Predict, Command, Observe, Verify

Connect setPower() and telemetry to real wheel behavior

25 MIN ACTIVITY	8 STUDENT ROLES	CONTROLLED TEST FORMAT	1 SAFETY LEAD
---------------------------	---------------------------	----------------------------------	-------------------------

ACTIVITY OUTCOME Students predict motion from left/right power pairs, identify object-method-argument syntax, and use telemetry plus observation to diagnose one mismatch at a time.

Materials

- Robot secured on stable blocks with both wheels clear.
- Prediction cards: 0.0/0.0, +0.3/+0.3, -0.3/-0.3, +0.3/-0.3, and +0.5/+0.2.
- Driver Station showing Left Power and Right Power telemetry.
- Printed predict-observe-verify sheet from this guide.
- POWERING ON safety poster visible at the test area.

Before students enter

- Verify configuration names and basic direction behavior from Lesson 2.
- Confirm both wheels have clearance through their complete path.
- Mark the robot's intended forward direction.
- Set the rule: every mismatch triggers STOP, DISABLED confirmation, and one-change-at-a-time diagnosis.

SAFETY BOUNDARY Students never cross into the robot test area while the OpMode is enabled. STOP and visually confirm DISABLED before anyone approaches.

Student setup and roles

Assign one primary role to each student. Rotate roles on a second trial if time permits. The safety lead may stop the activity at any time.

STUDENT	ROLE	RESPONSIBILITY
1	Safety lead	Runs the POWERING ON and approach checkpoints.
2	Driver Station	Controls INIT, PLAY, and STOP.
3	Driver	Applies one gentle command at a time.
4	Left observer	Reports left-wheel direction and relative speed.
5	Right observer	Reports right-wheel direction and relative speed.
6	Telemetry reader	Reads commanded Left Power and Right Power.
7	Recorder	Captures prediction, command, observation, and result.
8	Debugger	Chooses the next single check after a mismatch.

Ground rules

- Predict before testing; record evidence before proposing a correction.
- Only the Driver Station operator touches INIT, PLAY, or STOP.
- Any student may call STOP for unexpected motion, instability, heat, noise, or an unclear boundary.
- Change only one variable between tests so the evidence remains interpretable.

POWERING ON gate

SAY AND DO ALL SIX STEPS 1. Secure the robot. 2. Confirm DISABLED. 3. Look around. 4. Make sure everyone is clear. 5. Call POWERING ON! and wait. 6. Visually confirm clear, then enable/PLAY.

After every test: press STOP, visually confirm DISABLED, and only then permit anyone to approach the robot.

25-minute teacher walkthrough

1. Decode setPower() | 4 minutes

TEACHER	<i>"Display leftMotor.setPower(leftPower); and ask students to identify the object, method, and argument."</i>
---------	--

STUDENT	<i>"Students label leftMotor as object, setPower as method, and leftPower as argument/current requested value."</i>
---------	---

ASK	<i>"Which part contains the value being sent?"</i>
-----	--

LISTEN FOR	The argument inside parentheses: leftPower.
-------------------	---

TEACHING NOTE	Repeat with rightMotor so students see the same pattern applied to a different object.
----------------------	--

2. Predict from sign and magnitude | 5 minutes

TEACHER	<i>"Reveal one left/right pair at a time. Require a written prediction before any live command."</i>
---------	--

STUDENT	<i>"Students use sign for commanded direction and magnitude for relative strength, then predict straight, backward, stop, spin, or curve."</i>
---------	--

ASK	<i>"What should +0.3 and -0.3 do?"</i>
-----	--

LISTEN FOR	The sides are commanded in opposite directions, producing an in-place turn/spin if the drivetrain is configured correctly.
-------------------	--

TEACHING NOTE	Predictions assume both positive commands correspond to the team's physical forward convention.
----------------------	---

3. Test one side at a time | 6 minutes

TEACHER	<i>"Run the full POWERING ON sequence, press PLAY, move the left stick gently, center it, then move the right stick gently."</i>
STUDENT	<i>"Observers report from outside the boundary while telemetry reader reads the commanded values. Recorder compares each to the prediction."</i>
ASK	<i>"Why start with one side?"</i>
LISTEN FOR It isolates the relationship among one stick, one variable, one setPower call, one telemetry value, and one wheel.	
TEACHING NOTE If anything is unexpected, press STOP before discussing causes.	

4. Test combined commands | 6 minutes

TEACHER	<i>"Use gentle straight, backward, spin, and curve inputs. Pause at center between cases."</i>
---------	--

STUDENT	<i>"For each case, students say prediction, read telemetry, observe both wheels, and mark match or mismatch."</i>
---------	---

ASK	<i>"What does telemetry prove?"</i>
-----	-------------------------------------

LISTEN FOR It shows the values commanded by the program; it does not independently measure wheel speed or direction.

TEACHING NOTE Keep tests short. Heat, noise, or instability triggers immediate STOP.

5. Diagnose safely | 4 minutes

TEACHER	<i>"Press STOP and confirm DISABLED before the team approaches. Choose only one evidence-based check."</i>
---------	--

STUDENT	<i>"The debugger selects configuration name, direction setting, wiring, mechanical binding, or input/telemetry as the next check."</i>
---------	--

ASK	<i>"Why change only one thing?"</i>
-----	-------------------------------------

LISTEN FOR A single controlled change lets the team know which correction affected the result.

TEACHING NOTE Repeat the entire POWERING ON sequence before every retest.

Predict - command - observe - verify

Student/team: _____ Date: _____ Robot: _____

LEFT / RIGHT	PREDICTION	TELEMETRY + OBSERVATION	MATCH? / NEXT CHECK
0.0 / 0.0			
+0.3 / +0.3			
-0.3 / -0.3			
+0.3 / -0.3			
+0.5 / +0.2			
Student test: ____ / ____			
STUDENT EVIDENCE RULE Write what was commanded and what was observed. Do not write a correction until STOP has been pressed and DISABLED has been confirmed.			

Safety sign-off

Before live testing, the safety lead confirms: robot secure [] DISABLED [] boundary clear [] POWERING ON called and wait completed []

After testing: STOP pressed [] DISABLED visually confirmed [] Safety lead initials: _____

COMPLETION CRITERIA The student uses lesson vocabulary, predicts before testing, records evidence, explains one result from the code or math, and follows STOP/DISABLED rules before approaching the robot.

Debrief

ASK	LISTEN FOR
What does <code>setPower(0.0)</code> request?	No motor output, while the OpMode may continue running.
What do sign and magnitude represent?	Sign is commanded direction relative to configuration; magnitude is requested strength.
Why can telemetry and wheel motion disagree?	Telemetry shows the command, while mapping, direction configuration, wiring, or mechanics determine physical response.
What is the response to any mismatch?	STOP, confirm DISABLED, then diagnose one item at a time.

Troubleshooting without guessing

OBSERVATION	NEXT CHECK AFTER STOP/DISABLED
Telemetry correct; wheel direction wrong	Check motor Direction after STOP/DISABLED.
Telemetry correct; no wheel motion	Check mapping, wiring, hub connection, battery, and mechanical binding.
Left telemetry changes; right wheel moves	Recheck configuration-name-to-port mapping.
Robot shifts on stand	STOP immediately; improve support and clearance before any retest.

Technical notes for the teacher

- `setPower(double)` receives the program's requested motor output, normally within -1.0 to +1.0.
- Sign is relative to configured motor direction; mounting determines physical wheel motion.
- Equal values produce straight motion only when directions and mechanics are aligned.
- Telemetry reports commanded values, not an independent wheel-speed measurement.
- The runtime disables outputs after STOP, but the team must still visually confirm DISABLED before approach.

EXIT TICKET Telemetry shows +0.3 for a wheel that turns physically backward. List the first two facts you would verify after STOP and DISABLED.

HANDS-ON ACTIVITY FOCUS GUIDE

Mix and Normalize Split Arcade Drive

One stick drives, one stick turns

25 MIN ACTIVITY	8 STUDENT ROLES	MATH -> ROBOT FORMAT	1 SAFETY LEAD
---------------------------	---------------------------	-----------------------------------	-------------------------

ACTIVITY OUTCOME Students combine drive and turn inputs, recognize an out-of-range mix, normalize both outputs with the same denominator, and connect the result to robot motion.

Materials

- BasicArcadeDriveTeleOp.java or the five key split-arcade lines displayed.
- Role cards: DRIVE, TURN, ADD, SUBTRACT, DENOMINATOR, LEFT POWER, RIGHT POWER, and MOTION.
- Value cards including 0.0, +0.2, +0.5, +0.6, and +1.0.
- Printed mix-and-normalize worksheet from this guide.
- Optional secured robot with Drive, Turn, Left Power, and Right Power telemetry.

Before students enter

- Confirm the convention: positive drive requests forward; positive turn requests a right turn for this example.
- Write the left and right formulas where every student can see them.
- Prepare at least one case that does not need normalization and one that does.
- Keep the robot disabled until students complete the math cases successfully.

SAFETY BOUNDARY Students never cross into the robot test area while the OpMode is enabled. STOP and visually confirm DISABLED before anyone approaches.

Student setup and roles

Assign one primary role to each student. Rotate roles on a second trial if time permits. The safety lead may stop the activity at any time.

STUDENT	ROLE	RESPONSIBILITY
1	DRIVE	Holds <code>-gamepad1.left_stick_y</code> .
2	TURN	Holds <code>gamepad1.right_stick_x</code> .
3	ADD	Calculates drive + turn.
4	SUBTRACT	Calculates drive - turn.
5	DENOMINATOR	Calculates $\max(\text{abs}(\text{drive}) + \text{abs}(\text{turn}), 1.0)$.
6	LEFT POWER	Divides the left raw mix by the denominator.
7	RIGHT POWER	Divides the right raw mix by the denominator.
8	MOTION	Predicts the resulting robot movement.

Ground rules

- Predict before testing; record evidence before proposing a correction.
- Only the Driver Station operator touches INIT, PLAY, or STOP.
- Any student may call STOP for unexpected motion, instability, heat, noise, or an unclear boundary.
- Change only one variable between tests so the evidence remains interpretable.

POWERING ON gate

SAY AND DO ALL SIX STEPS 1. Secure the robot. 2. Confirm DISABLED. 3. Look around. 4. Make sure everyone is clear. 5. Call POWERING ON! and wait. 6. Visually confirm clear, then enable/PLAY.

After every test: press STOP, visually confirm DISABLED, and only then permit anyone to approach the robot.

25-minute teacher walkthrough

1. Compare control styles | 3 minutes

TEACHER	"Ask students to show with their hands how tank drive and split arcade assign joystick jobs."
---------	---

STUDENT	"Students state: tank uses one Y stick per side; split arcade uses left Y for drive and right X for turn."
---------	--

ASK	"Why is this called split arcade?"
-----	------------------------------------

LISTEN FOR	The forward/back and turning controls are split across two sticks.
-------------------	--

TEACHING NOTE	Do not call it single-stick arcade; that usually places both axes on one stick.
----------------------	---

2. Mix a straight command | 4 minutes

TEACHER	"Set drive = +0.5 and turn = 0.0. Walk through raw left, raw right, denominator, and final outputs."
---------	--

STUDENT	"ADD and SUBTRACT both produce +0.5; DENOMINATOR produces 1.0; both power roles show +0.5; MOTION predicts straight forward."
---------	---

ASK	"Why does division by 1.0 leave the outputs unchanged?"
-----	---

LISTEN FOR	Values already in range should not be amplified or reduced.
-------------------	---

3. Mix a pure turn | 4 minutes

TEACHER	<i>"Set drive = 0.0 and turn = +0.5. Require both formulas to be evaluated."</i>
----------------	--

STUDENT	<i>"Left raw mix is +0.5; right raw mix is -0.5; denominator is 1.0; MOTION predicts a right in-place turn under this convention."</i>
----------------	--

ASK	<i>"Why do the formulas use opposite turn signs?"</i>
------------	---

LISTEN FOR	One side must increase relative to the other to create steering.
-------------------	--

TEACHING NOTE	Connect the sign pattern to the earlier tank-drive motion lesson.
----------------------	---

4. Normalize an out-of-range mix | 7 minutes

TEACHER	<i>"Set drive = +1.0 and turn = +1.0. Do not skip the raw-mix step."</i>
----------------	--

STUDENT	<i>"ADD shows +2.0; SUBTRACT shows 0.0; DENOMINATOR shows 2.0; final powers become +1.0 and 0.0."</i>
----------------	---

ASK	<i>"Why divide both sides by the same number?"</i>
------------	--

LISTEN FOR	It brings both into range while preserving the ratio that creates the intended steering.
-------------------	--

TEACHING NOTE	Normalization is not clamping each side independently; independent clipping can distort steering.
----------------------	---

5. Student case and safe transfer | 7 minutes

TEACHER	"Students solve drive = +0.6, turn = +0.2. After checking +0.8/+0.4 with denominator 1.0, optionally transfer to the secured robot."
STUDENT	"Students predict a gentle right curve. For a live test, the team completes POWERING ON, tests one small command, presses STOP, and confirms DISABLED."
ASK	"What should telemetry show for the solved case?"
LISTEN FOR Drive +0.6, Turn +0.2, Left Power +0.8, Right Power +0.4.	
TEACHING NOTE Use smaller physical stick inputs than the full-value paper examples.	

Split-arcade mixing and normalization

Student/team: _____ Date: _____ Robot: _____

DRIVE / TURN	RAW LEFT / RIGHT	DENOMINATOR	FINAL L / R + MOTION
+0.5 / 0.0			
0.0 / +0.5			
+0.6 / +0.2			
+1.0 / +1.0			
-0.5 / 0.0			
Student case: ____ / ____			
STUDENT EVIDENCE RULE Write what was commanded and what was observed. Do not write a correction until STOP has been pressed and DISABLED has been confirmed.			

Safety sign-off

Before live testing, the safety lead confirms: robot secure [] DISABLED [] boundary clear [] POWERING ON called and wait completed []

After testing: STOP pressed [] DISABLED visually confirmed [] Safety lead initials: _____

COMPLETION CRITERIA The student uses lesson vocabulary, predicts before testing, records evidence, explains one result from the code or math, and follows STOP/DISABLED rules before approaching the robot.

Debrief

ASK	LISTEN FOR
What are the two driver inputs?	Left-stick Y supplies drive after negation; right-stick X supplies turn.
Why can raw mixes reach magnitude 2.0?	Drive and turn can each approach magnitude 1.0 and are added on one side.
What does the denominator guarantee?	It is at least 1.0 and scales out-of-range mixes back into the motor-power range.
What relationship must normalization preserve?	The ratio between left and right outputs, which preserves steering intent.

Troubleshooting without guessing

OBSERVATION	NEXT CHECK AFTER STOP/DISABLED
Robot turns opposite the lesson prediction	After STOP/DISABLED, verify turn sign and motor direction configuration.
One output exceeds 1.0 on paper	Calculate $\text{abs}(\text{drive}) + \text{abs}(\text{turn})$, use at least 1.0, then divide both raw outputs.
Student divides only the larger side	Require one shared denominator for both left and right outputs.
Straight command curves	Check direction, mechanical drag, wheel condition, and side-to-side output evidence.

Technical notes for the teacher

- This is split/two-stick arcade drive; single-stick arcade uses both axes of one stick.
- $\text{leftPower} = (\text{drive} + \text{turn}) / \text{denominator}$; $\text{rightPower} = (\text{drive} - \text{turn}) / \text{denominator}$.
- $\text{max}(\text{abs}(\text{drive}) + \text{abs}(\text{turn}), 1.0)$ avoids amplifying ordinary in-range values.
- Dividing both sides by the same denominator preserves the steering ratio.
- The required reversed motor side depends on the physical drivetrain, not the drive style's name.

EXIT TICKET For drive = +0.6 and turn = +0.2, calculate leftPower and rightPower and state whether normalization is needed.